

1. INTRODUCTION

This dissertation provides a comprehensive overview of the design and development of a fully functional, console-based Banking System application, engineered exclusively using the Java programming language. The dissertation exemplifies the practical application of core software engineering principles within a constrained yet powerful environment. The principal aim of this initiative is to construct a simplified but robust software model that accurately mirrors the essential operational mechanics of a modern banking institution. By simulating critical financial transactions, the system provides a clear window into the logic that underpins digital banking services.

A key driving force behind this dissertation is the need to demonstrate how software can effectively mitigate common operational risks associated with manual financial handling, such as calculation inaccuracies, procedural inconsistencies, and the lack of immediate validation. This system is meticulously architected around a layered design pattern, which enforces a clear separation of concerns. This structure is fundamental to organizing the codebase into distinct modules for user interaction, application control, business logic, and data handling. The User Layer is the text-based console, serving as the exclusive point of interaction between the customer and the banking services. It relies on Java's Scanner class for robust input handling and System.out for presenting a clear, menu-driven interface and transaction outputs.

Directing the user's journey is the Application Layer, typically implemented in a Main or Banking class. This layer functions as the application's conductor, managing the main program loop, displaying menus, interpreting user choices, and routing requests to the appropriate services. The heart of the system's functionality resides in the Business Logic Layer, encapsulated within a Bank Account class or similar entity. This layer is the custodian of all rules and operations, including the execution of deposits and withdrawals and the maintenance of an accurate running balance.

Integral to the Business Logic Layer is a suite of stringent Validation Rules. These rules are programmed to automatically reject invalid transactions, such as withdrawals that would result in a negative balance or deposits with non-positive values, ensuring the account's financial integrity is never compromised. For data persistence, the system employs an In-Memory Data Layer. All account information, including identifiers, holder names, and balances, is stored within the live memory of the Java Virtual Machine (JVM) as object states, making it volatile and session-specific. The technological foundation of this dissertation is purposefully minimalistic, leveraging only the standard libraries provided within the Java.

The deliberate choice to utilize transient, in-memory storage prioritizes conceptual clarity and simplicity for this academic exercise. It allows the dissertation to focus purely on transaction processing logic without the added complexity of database integration or file I/O operations. The technological foundation of this dissertation is purposefully minimalistic, leveraging only the standard libraries provided within the Java Development Kit (JDK). This demonstrates the inherent power and self-sufficiency of the Java platform for building logical, stand-alone applications.

The intended primary audience for this system includes computer science students and junior developers. It acts as an excellent practical tool for mastering object-oriented programming concepts, basic input/output operations, and control flow in a realistic context. While the feature set is intentionally narrow, the dissertation successfully encapsulates the complete cycle of fundamental banking transactions. It demonstrates a seamless flow from user input through processing to output, forming a cohesive and functional unit. The defined scope of this work includes the complete software development process, from the initial requirement gathering and system design to the hands-on implementation, thorough unit testing, and the creation of this accompanying documentation.

A significant challenge addressed in this dissertation is designing an intuitive user experience within the limitations of a console. The menu system must be logically structured and provide clear prompts and feedback to guide the user effectively without a graphical interface. Furthermore, the dissertation highlights the importance of data integrity within financial systems. Every transaction is processed through conditional checks within the business logic, ensuring that the account's state is always valid and consistent.

This application also serves as a foundational prototype. Its modular design means that individual layers, such as the in-memory data storage, could be replaced in the future with more persistent solutions like a database, without requiring a complete rewrite of the business logic. From an educational perspective, this dissertation and its associated codebase provide a tangible example of how abstract programming concepts can be synthesized to create a working application that solves a defined problem in a specific domain.

The following chapters will delve into the detailed design specifications, including UML diagrams depicting the class structure, a thorough explanation of the algorithms used, and the results of the testing phases that verify the system's correctness and robustness. This console-based Banking System stands as a testament to the effectiveness of Java and structured programming. It provides a fully operational, educational tool that lays the groundwork for understanding more complex financial software systems.

Algorithm

The functionality of the Banking System is governed by a set of core algorithms that define the logical sequence of operations for processing user requests. These algorithms are implemented within the methods of the Bank Account class and orchestrated by the main application controller. Their precision is critical to ensuring the financial integrity and reliability of the system. The most fundamental algorithm within the system is the Transaction Processing Algorithm, which handles both deposits and withdrawals. This algorithm is designed to be atomic, meaning it must execute completely and successfully or not at all, leaving the account data in a consistent state. It is triggered by a user action from the console menu.

The algorithm begins by accepting an input value from the user via the application controller. This value, representing the transaction amount, is passed as a parameter to the respective method (deposit or withdraw) in the Bank Account class. The first step within the method is input validation. The validation sub-algorithm checks if the provided amount is a numerical value greater than zero. This is a crucial guard clause that prevents logically impossible transactions, such as depositing a negative value or withdrawing zero funds. If this validation fails, the algorithm terminates immediately, and an error message is returned to the user interface layer.

For a withdrawal request, a second validation check is executed: sufficient funds verification. The algorithm compares the requested withdrawal amount against the current balance attribute of the account object. If the amount exceeds the available balance, the transaction is aborted, and an "insufficient funds" error is generated. Only after both validation checks are passed does the algorithm proceed to modify the account state. For a deposit, the amount is added to the current balance. For a withdrawal, the amount is subtracted from the current balance. This operation is a straightforward arithmetic calculation but represents the core financial action of the system.

Following the successful update of the balance, the algorithm generates an output. This typically involves returning a success confirmation message to the controller, which is then displayed to the user. The message includes details of the transaction and the updated account balance, providing immediate feedback and reinforcing the action's completion. The Balance Inquiry Algorithm is more straightforward. It involves no state modification. Its sole function is to retrieve the current value of the balance attribute from the Bank Account object and format it into a readable message for display on the console.

The Banking System is a self-contained, console-based application. Its configuration is

defined by its software architecture, development environment, and runtime specifications, all centred on the Java platform. The system is designed for simplicity, portability, and ease of execution, requiring minimal setup and external dependencies.

Architectural Overview

The system adheres to a structured, multi-layered architectural pattern, implemented entirely within a single Java Virtual Machine (JVM) process. This design promotes separation of concerns, making the code modular, testable, and maintainable.

The application is built using a minimal and focused technology stack, leveraging only the core libraries provided by the Java Standard Edition (SE) platform. Programming Language: Java 17 (JDK 17 or later). The use of a Long-Term Support (LTS) version ensures stability and access to modern language features.

Key Java Packages & Classes

- **java.util.Scanner:** For parsing primitive types and strings from the standard input stream (System.in).
- **java.lang.System:** Utilizes System.out for printing text to the console and System.in for receiving input.
- **Core Language Features:** The dissertation utilizes fundamental OOP concepts (Classes, Objects, Encapsulation), control flow statements (loops, conditionals), and primitive data types (double for balance, String for names, long for account numbers).

Development Environment

- **Integrated Development Environment (IDE):** Any Java-capable IDE is sufficient (e.g., IntelliJ IDEA, Eclipse, VS Code with Java extensions). The dissertation is not dependent on any IDE-specific features.
- **Build Tool:** The application can be compiled and run directly using the command-line tools (javac, java) provided with the JDK. No external build tools like Maven or Gradle are required for basic functionality, though they could be incorporated for dissertation management.
- **Data Storage:** Data persistence is achieved in-memory through the state of instantiated Java objects (e.g., instances of Bank Account). All data is volatile and exists only for the duration of the JVM's execution.

1.1 SYSTEM CONFIGURATION

1.2 HARDWARE CONFIGURATION

Processor	:	Intel Core i7 or higher (or equivalent AMD processor)
RAM	:	Minimum 4 GB (8 GB recommended for smoother multitasking)
Display	:	1024 × 768 resolution or higher
Input Devices	:	Standard keyboard and mouse
Other Requirements	:	Stable power supply and optional UPS for uninterrupted operation

1.3 SOFTWARE CONFIGURATION

Operating System	:	Windows 10/11, Linux (Ubuntu/Fedora), or macOS
Programming Language	:	Java SE (Standard Edition)
JDK Version	:	Java Development Kit (JDK) 8 or later (JDK 24 recommended)
IDE/Editor (Optional)	:	Eclipse, IntelliJ IDEA, NetBeans, or any text editor (e.g., VS Code, Notepad++)
Runtime Environment	:	Java Runtime Environment (JRE) compatible with the installed JDK
Command-Line Tool	:	Terminal/Command Prompt for compiling and executing the Program.

2. SYSTEM STUDY

2.1 EXISTING SYSTEM

The existing manual banking process relies heavily on paperwork and in-person transactions. Account creation, deposits, and withdrawals require physical presence at a bank branch. Data management is time-consuming and prone to human errors.

2.1.1 DRAWBACKS

- The manual banking process is slow and requires customers to visit the branch for every transaction.
- Paper-based record keeping makes data management time-consuming and difficult to update accurately.
- Human errors such as miscalculations or misplaced documents can lead to incorrect account balances.

2.2 PROPOSED SYSTEM

The proposed Java-based Banking System provides a fully automated solution for basic banking operations. It allows users to create accounts, deposit money, withdraw funds, and check balances instantly through a console interface. A double-account mode supports direct fund transfers with proper validation. By applying object-oriented principles, the system ensures secure data handling and modular design. Real-time input through Java's Scanner class makes the application interactive and user-friendly. This digital approach saves time and reduces the risk of human error.

2.2.1 FEATURES

The proposed system enables users to create new accounts or access predefined ones with ease. It supports secure deposits, withdrawals, and instant balance checks through a simple console menu. A double-account mode allows validated fund transfers between two accounts in real time. Object-oriented design ensures data security, modularity, and easy maintenance of the code. Interactive input using Java's Scanner class makes transactions quick, accurate, and user-friendly.

3. SYSTEM DESIGN AND DEVELOPMENT

This chapter details the architectural blueprint and the implementation strategy of the Console-Based Banking System. It translates the requirements and features into a concrete software design, focusing on the object-oriented structure, the core algorithms, and the development process that brings the system to life.

System Architecture

The system follows a layered, modular architecture implemented within a single Java application. This design promotes separation of concerns, making each component responsible for a distinct part of the functionality.

Component Description

- **User Interface (UI) Layer:** This is the presentation layer, solely responsible for all interactions with the user. It uses `System.out.println()` statements to display text, menus, and results on the console. It uses the `Scanner` class to read and parse user input from `System.in`.
- **Application Control Layer:** This layer, typically contained within the main method, acts as the controller. It contains the main program loop, presents the UI menu, captures the user's choice, and based on that choice, calls the appropriate method on the `BankAccount` object. It handles the flow of the application but not the business logic itself.
- **Business Logic Layer:** This is the core computational engine of the system, implemented in the `BankAccount` class. It contains the methods that enforce the rules of the bank (e.g., "cannot withdraw more than the balance") and perform the operations on the data (e.g., modifying the balance). It is completely independent of the UI.
- **Data Storage Layer:** This is not a separate database but the aggregated state of the objects in memory. The data (account holder's name, account number, and balance) is stored as private attributes within instances of the `BankAccount` class. This encapsulates and protects the data, allowing access only through the public methods of the Business Logic Layer. All account information, including identifiers, holder names, and balances, is stored within the live memory of the Java Virtual Machine (JVM) as object states, making it volatile and session-specific.

BankAccount Class

- **Purpose:** To model a real-world bank account, encapsulating its data and behaviors into a single, cohesive unit.
- **Attributes (All private):**
 1. `accountNumber` (long or String): The unique identifier for the account.
 2. `accountHolderName` (String): The full name of the account owner.
 3. `balance` (double): The current amount of money in the account.
- **Key Methods (public):**
 1. `BankAccount(String name, long accNum, double initialDeposit)`: Constructor to initialize a new account object.
 2. `deposit(double amount)`: Adds the specified amount to the balance if the amount is valid (> 0).
 3. `withdraw(double amount)`: Subtracts the specified amount from the balance if the amount is valid and sufficient funds exist.
 4. `getBalance()`: Returns the current balance. A classic getter method.
 5. `displayAccountInfo()`: Prints the account details (number, name, balance) to the console.

Main Application Loop Algorithm

1. Initialize a Scanner object and a BankAccount object.
2. Display the main menu.
3. Read the user's choice.
4. Use a switch statement on the choice:
 - Case 1: Prompt for amount and call `account.deposit(amount)`.
 - Case 2: Prompt for amount and call `account.withdraw(amount)`.
 - Case 3: Call `account.getBalance()` and display the result.
 - Case 4: Print a goodbye message and break the loop.
 - Default: Print "Invalid option."
5. If the choice was not 4 (Exit), loop back to Step 2.

Withdrawal Method Algorithm

1. Input: double amount
2. Process:
 - If `amount <= 0`, print "Error: Amount must be positive." and exit.

- Else if amount > balance, print "Error: Insufficient funds." and exit.
 - Else, subtract amount from balance.
3. Output: Print "Withdrawal successful. New balance: [balance]".

Development Approach

The system was developed using an iterative and incremental approach.

1. Core Model First: The BankAccount class was implemented first with its attributes and method stubs, establishing the data structure.
2. Logic Implementation: The business logic for deposit and withdrawal was then coded, including validation checks.
3. UI and Integration: The main method and console interface were developed to interact with the BankAccount object.
4. Testing and Refinement: Each method was tested rigorously with valid and invalid inputs. The UI flow was refined for clarity, and error messages were improved based on testing feedback.
5. Feature Addition: Additional features like transaction history were considered and could be added as a List<String> in the BankAccount class in subsequent iterations.

3.1 FILE DESIGN

In the context of this console-based Banking System, "file design" refers to the structure and organization of the Java source code files and the logical structure of data within the application. Since the system uses in-memory data storage and does not interact with external files or databases, the focus is on the .java files that constitute the program and the internal data handling.

3.1.1. SOURCE CODE FILE STRUCTURE

The application is comprised of Java source code files (.java), each containing a specific class that defines a component of the system. The structure is simple and modular.

File Descriptions

1. Main.java

- Purpose: This file contains the entry point of the application, the main method. It is responsible for the application's control flow and user interaction.

Contents

- The main(String[] args) method.
- Instantiation of the Scanner object for console input.
- Instantiation of a BankAccount object.
- The main program loop with a while or do-while statement.
- A switch statement to handle the user's menu selection.
- Calls to the methods of the BankAccount object (e.g., deposit(), withdraw()).
- System.out statements for displaying menus, prompts, and results.
- Basic input validation for menu choices.

2. BankAccount.java

- Purpose: This file contains the definition of the BankAccount class. It encapsulates the core data and business logic of the banking system.

Contents

- Private Attributes (Instance Variables): private long accountNumber;, private String accountHolderName;, private double balance;.
- Constructor: public BankAccount(long accNum, String name, double initialBalance) for initializing new account objects.

Public Methods

- The key methods that define the account's behavior:
- public void deposit(double amount)
- public void withdraw(double amount)
- public double getBalance()
- public void displayAccountDetails()

Validation Logic

Code within the deposit and withdraw methods to check for non-positive amounts and insufficient funds.

3.1.2 INTERNAL DATA STRUCTURE

While not persistent files, the system manages data using specific Java structures within the runtime memory.

Data Handling

- **Primary Data Storage:** Data is stored as the state of objects. An instance of the `BankAccount` class holds all information for a single account in its attributes (`accountNumber`, `accountHolderName`, `balance`).

Data Type Selection

- `long` for `accountNumber`: Chosen for its ability to hold large numerical values.
- `String` for `accountHolderName`: Chosen as the appropriate type for textual data.
- `double` for `balance`: Chosen for its ability to represent decimal numbers, which is essential for financial calculations involving cents.
- **Data Lifecycle:** The data is volatile. When the application is run, a `BankAccount` object is created, and its data lives in the RAM. When the application is terminated, the JVM stops, and all data held by the object is lost. This is a deliberate design choice for this specific dissertation scope.

3.1.3 TRANSACTION LOGGING (POTENTIAL ENHANCEMENT)

A logical extension for persistent storage would be the introduction of a file-based transaction log. This would involve:

- **New File:** `transaction.log`
- **Format:** Each line in the file could represent a transaction in a structured format, such as CSV:
- `timestamp,accountNumber,transactionType,amount,newBalance`
- **Example:** `2023-10-26T14:30:01,10001,DEPOSIT,500.00,1500.00`
- **Mechanism:** The deposit and withdraw methods in the `BankAccount` class would be modified to append a new line to this log file after every successful transaction. This file would act as a simple audit trail.

3.2 INPUT DESIGN

Input Design is a critical phase that defines how data enters the system. A well-designed input mechanism ensures data accuracy, simplicity, efficiency while minimizing the potential for errors. For this console-based Banking System, input design focuses on the strategies for capturing user commands and transaction data through the text-based interface. The input is validated to ensure it is an integer and that it corresponds to one of the available menu options (e.g., 1, 2, 3, 4). Invalid choices trigger an error message and a re-display of the menu.

3.2.1 OBJECTIVES OF INPUT DESIGN

The primary objectives guiding the input design for this system are:

- Accuracy: To ensure that the data entered is correct, valid, and conforms to expected formats (e.g., numerical amounts for money).
- Simplicity and Ease of Use: To make the input process straightforward and intuitive for the user, reducing the learning curve and preventing confusion.
- Error Prevention and Handling: To minimize input errors through validation and to provide clear, helpful feedback when errors do occur, guiding the user to correct them.
- Consistency: To maintain a uniform style and flow for all input operations throughout the application.
- Efficiency: To allow for the quick and efficient entry of data.

3.2.2 INPUT METHODS AND DEVICES

The system relies on a single, standard method for all data entry:

- Method: Text-based input via the console.
- Device: Standard keyboard. The user enters data by typing on the keyboard and finalizing the entry with the 'Enter' key.
- Java Mechanism: The `java.util.Scanner` class is the primary tool for reading input from the standard input stream (`System.in`). Different Scanner methods like `nextInt()`, `nextDouble()`, and `nextLine()` are used based on the type of data required.

3.2.3 INPUT TYPES AND SPECIFICATIONS

The system handles two main categories of input: control input (menu navigation) and data input (transaction details).

Control Input (Menu Selection)

- Purpose: To allow the user to navigate the application menu and select the desired operation (e.g., Deposit, Withdraw, Exit).

Design

- A numbered list of options is displayed prominently.
- Prompt: "Enter your choice: "
- Data Type: Integer (int).
- Validation: The input is validated to ensure it is an integer and that it corresponds to one of the available menu options (e.g., 1, 2, 3, 4).

3.2.4 DATA INPUT (TRANSACTION AMOUNTS)

- Purpose: To capture the monetary value for deposit and withdrawal transactions.

Design

- A clear prompt specifies what is needed.
- Prompt: "Enter amount to deposit: " or "Enter amount to withdraw: "
- Data Type: Double (double) or Big Decimal (for precise financial calculations, though double is used here for simplicity).
- Validation: This is the most critical validation step. The system checks:
 1. Data Type Validation: Ensures the input is a valid number (e.g., prevents crashes if the user enters "abc").
 2. Business Rule Validation: Ensures the number is greater than zero. A negative or zero amount is rejected as invalid for a transaction.

3.2.5 DATA INPUT (ACCOUNT CREATION)

- Purpose: To capture initial customer data when creating a new account (if this feature is implemented).

Design

- Prompt for Name: "Enter account holder name: "
- Data Type: String (String). Validation can be added to check for non-empty input.
- Prompt for Account Number: "Enter account number: "
- Data Type: Long (long) or String (String). Validation ensures it is a positive number (if using long) or a non-empty string.
- Prompt for Initial Balance: "Enter initial deposit: "
- Data Type: Double (double). Validation ensures the amount is zero or greater.

3.2.6 INPUT VALIDATION AND ERROR HANDLING

A robust validation mechanism is integrated into the input process:

- Try-Catch Blocks: Input operations for numbers are wrapped in try-catch blocks to handle InputMismatchException. This gracefully catches non-numerical input when a number is expected.
- Conditional Checks: After successful reading, values are checked against business rules (e.g., if(amount > 0)).

- Error Feedback: When validation fails, a clear, user-friendly error message is displayed instead of a technical exception stack trace.
 1. Example: "Invalid input. Please enter a number."
 2. Example: "Error: Amount must be greater than zero."
 3. Example: "Invalid menu option. Please choose between 1 and 4."
- Recovery Mechanism: After an error, the control typically returns to the main menu or re-prompts the user for the specific input, allowing them to correct the mistake without crashing the application.

3.3 OUTPUT DESIGN

Output Design is the process of defining the format, content, and presentation of the information that the system presents to the user. Effective output design is crucial as it is the primary channel through which the system communicates results, confirms actions, and provides feedback. For a console-based system, output must be clear, concise, and well-formatted to ensure a positive and efficient user experience.

Objectives Of Output Design

The output design for this Banking System is guided by the following key objectives:

- Clarity and Readability: Information must be presented in a clear, unambiguous, and easy-to-read manner. The user should be able to understand the system's response at a glance.
- Relevance: The output must be directly relevant to the user's request, providing exactly the information needed without unnecessary clutter.
- Timeliness: All output must be generated and displayed immediately after a user action, providing instant feedback.
- Consistency: The format, style, and terminology used across all outputs must be consistent throughout the application. This builds user familiarity and confidence.
- User-Friendly: Messages, especially errors, should be constructive and guide the user toward the correct next action.

Output Methods and Media

The system utilizes a single, straightforward method for all output:

- Method: Text-based output displayed on the console (standard output stream).

- Media: Computer monitor or terminal screen.
- Java Mechanism: The primary method for generating output is `System.out.println()` and `System.out.print()`, which print formatted text lines to the console.

3.3.1 OUTPUT TYPES AND SPECIFICATIONS

The system generates several distinct types of output; each designed for a specific purpose.

Navigation And Instructional Prompts (Menu)

- Purpose: To guide the user by presenting available options and prompting for input.

Design & Format:

- Uses a clean, numbered list format.
- Uses blank lines to separate the menu from other text for visual clarity.

Transaction Confirmation Messages

- Purpose: To confirm that a user's transaction request (deposit or withdrawal) was successful and to report the new account state.

Design & Format:

- Clearly states the action performed.
- Always confirms the transaction amount.
- Always displays the updated balance.

Example:

- Deposit successful. Amount \$500.00 added. New Balance: \$1,500.00

Informational Output (Balance Inquiry)

- Purpose: To present requested information to the user.

Design & Format:

- The output is a single, focused statement.
- The key data (the balance) is prominently displayed.

Example:

- Your current account balance is: \$1,500.00
- User-Friendly: Messages, especially errors, should be constructive and guide the user toward the correct next action.

3.3.2 ERROR AND VALIDATION MESSAGES

- Purpose: To inform the user of an invalid input or an impossible operation and to guide them toward a correction.

Design & Format:

- Tone: Helpful and instructive, not technical or accusatory.
- Content: Clearly states what went wrong and often what the expected input should be.
- Format: Often prefixed with "Error:" or "Invalid" to attract attention.

Examples:

- Error: Invalid menu option. Please select a number between 1 and 4.
- Error: Invalid amount. Please enter a positive number.
- Error: Transaction failed. Insufficient funds.

System Status Messages

- Purpose: To keep the user informed about the application's state.

Design & Format:

- Simple, friendly messages.

Examples:

- (On startup): Banking System initialized.
- (On exit): Thank you for using Banking System. Goodbye!

Formatting And Presentation

To enhance readability, the output employs simple formatting techniques:

- Currency Formatting: Balances and monetary amounts are formatted to two decimal places using `String.format("%.2f", balance)` or `NumberFormat.getCurrencyInstance()` to appear as standard currency values (e.g., \$100.50 instead of 100.5).
- Whitespace: Blank lines (`System.out.println()`) are used strategically to separate logical blocks of output (e.g., separating the menu from a transaction result), preventing the console from appearing as a "wall of text."
- Labels: Data values are always presented with a descriptive label (e.g., "New Balance: \$1500.00" instead of just "1500.00").

3.4 SYSTEM DEVELOPMENT

The System Development chapter documents the practical implementation of the Banking System. It details the transition from design specifications to a fully functional

software product, describing the development environment, the coding process, and the specific Java constructs used to bring the system to life.

Development Environment

The system was developed in a standardized environment to ensure consistency and reproducibility.

- **Programming Language:** Java SE 17 (JDK 17)
- **Integrated Development Environment (IDE):** IntelliJ IDEA Community Edition. The choice of IDE provided powerful features like syntax highlighting, code completion, and integrated debugging, which significantly accelerated development.
- **Build and Execution:** The application was compiled and executed directly from the IDE. Alternatively, it can be compiled using the `javac` command-line tool and run with the `java` command, demonstrating its independence from any specific development tool.
- **Version Control:** While not mandatory for a dissertation of this scale, basic version control principles were followed. The source code files (`Main.java`, `BankAccount.java`) could be managed using a system like Git to track changes and maintain a development history.

Key Programming Concepts Utilized

The development process effectively applied fundamental Java and software engineering principles:

- **Object-Oriented Programming (OOP):** The system is built around the concept of objects. The `BankAccount` class is a blueprint for creating account objects that encapsulate both data and behavior.
- **Encapsulation:** The core data (`balance`, `accountNumber`) is declared private. It can only be accessed or modified through public methods (`deposit`, `withdraw`, `getBalance`), protecting the data from invalid external changes.
- **Modularity:** The clear separation between the `Main` class (application flow) and the `BankAccount` class (business logic) makes the code organized, easier to debug, and simple to maintain. Each class has a single, well-defined responsibility.
- **Control Structures:** Extensive use of do-while loops, switch statements, and if-else conditions controls the flow of the program and implements the business logic.
- **Input/Output (I/O) Handling:** The `Scanner` class is used for robust input handling, and `System.out` is used for displaying formatted output to the console.

3.4.1 DESCRIPTION OF MODULES

The Banking System application is architected using a modular design. A module is a self-contained unit of code with a specific responsibility. This approach enhances the software's organization, readability, maintainability, and testability

Application Controller Module (Main Class)

This module is responsible for orchestrating the overall flow of the application and managing all user interaction. It acts as the conductor, coordinating between the user and the core banking logic.

- Purpose: To serve as the entry point of the application and to control the navigation, input/output handling, and menu presentation.

Key Responsibilities

- Application Initialization: Bootstrapping the application by creating necessary objects (e.g., Scanner, BankAccount).
- User Interface (UI) Management: Displaying the text-based menu and prompts to the user on the console.
- Input Handling: Reading and parsing all user input from the keyboard using the Scanner object.
- Program Flow Control: Using a loop and a switch-case structure to interpret the user's menu choice and keep the application running until an exit command is received.
- Service Delegation: Acting as a client to the BankAccount module. It does not perform banking calculations itself but delegates all requests (deposit, withdraw, etc.) to the appropriate methods of the BankAccount object.
- Output Presentation: Receiving results from the BankAccount module and displaying them to the user, including transaction confirmations and error messages.
- Resource Management: Properly closing the Scanner object upon application termination to prevent resource leaks.

Interactions

- This module is dependent on the BankAccount module. It calls its public methods (deposit, withdraw, getBalance) to perform operations.
- It interacts with the user through the console (System.in and System.out). Displaying the text-based menu and prompts to the user on the console.

3.4.2 BUSINESS LOGIC MODULE (BANKACCOUNT CLASS)

This module is the computational engine of the system. It encapsulates the core data and rules that define a bank account and what can be done with it.

- Purpose: To model a real-world bank account by storing its data and providing methods that enforce banking rules for operating on that data.

Key Responsibilities

- Data Encapsulation: Securely storing the account's critical data (account number, holder name, balance) as private instance variables, preventing direct external access.
- Business Rule Enforcement: Implementing and enforcing the fundamental rules of banking within its methods. This includes:
 - Ensuring deposit and withdrawal amounts are positive.
 - Ensuring a withdrawal does not exceed the available account balance (preventing overdrafts).
- Core Operation Execution: Performing the actual arithmetic operations of adding funds to the balance (deposit) and subtracting funds from the balance (withdrawal) once all validations are passed.
- State Provision: Providing a controlled public interface (a getter method) for other parts of the system, like the Main class, to read the current balance without being able to modify it directly.

Interactions

- This module is independent and reusable. It has no knowledge of the Main class or how it is used (console, GUI, etc.). This makes it highly portable.
- It is used by the Main module. It receives requests from the Main class, executes the validated logic, and returns results (or messages) back to it.

3.4.3 INTER-MODULE COMMUNICATION

The interaction between these two modules is a classic example of separation of concerns. The Main class handles the how of user interaction, while the BankAccount class handles the what of banking operations. This relationship can be visualized in the following sequence for a withdrawal:

- Main Class: Displays menu and captures user's choice (2 for Withdraw).
- Main Class: Prompts user for an amount and captures the input.

- Main Class: Calls `myAccount.withdraw(enteredAmount)`.
- BankAccount Class: The `withdraw` method validates the amount.
- BankAccount Class: If valid, it updates the balance; if invalid, it generates an error message.
- BankAccount Class: The method prints a result message to the console.
- Control returns to Main Class: The main loop continues, and the menu is displayed again.

Step-by-Step Explanation:

1. User Input: The user interacts with the console menu provided by the Main class, selecting the `withdraw` option and entering an amount.

2. Request Initiation: The Main class, acting as the controller, captures the input. It then initiates communication by calling the `withdraw` method on the BankAccount object, passing the entered amount as a parameter: `myAccount.withdraw(200.0)`;

3. Request Processing: Control is transferred to the BankAccount module. The method executes its internal sequence:

- Validation: It checks the business rules (is the amount positive? is it less than or equal to the balance?).
- State Change: If the checks pass, it updates the private balance variable.
- Notification: It generates a success message and prints it directly to the console using `System.out.println(...)`.

4. Response / Acknowledgement: The method completes its execution, and control returns to the Main class. The response, in this case, is not a return value but the side effect of the action (the updated balance and the console message). The Main class then returns to its main loop, ready for the next user command.

Data Flow

- Control Flow: The Main class is always the initiator. The BankAccount class is reactive; it only acts when one of its methods is called.
- To BankAccount: Data flows from Main to BankAccount via method parameters (the transaction amounts).
- From BankAccount: Data flows back to the Main class via return values (like the balance from `getBalance()`) and indirectly through the console output.

4. TESTING AND IMPLEMENTATION

The Testing and Implementation phase is critical to ensuring that the developed software is reliable, functional, and meets its specified requirements. This chapter details the strategies used to verify the correctness of the Banking System and the process of deploying it into its intended environment.

4.1 TESTING STRATEGY

A multi-faceted testing approach was employed to validate the system, focusing on unit testing, integration testing, and user acceptance testing.

4.1.1 UNIT TESTING

Unit testing involved verifying the smallest testable parts of the application—the individual methods of the BankAccount class—in isolation.

Deposit(double amount) method:

- Test Case 1 (Valid Positive Deposit): Input: deposit(500.0). Expected: Balance increases by 500.0. Result: PASS
- Test Case 2 (Invalid Negative Deposit): Input: deposit(-100.0). Expected: Error message, balance unchanged. Result: PASS
- Test Case 3 (Invalid Zero Deposit): Input: deposit(0.0). Expected: Error message, balance unchanged. Result: PASS

Withdraw(double amount) method:

- Test Case 1 (Valid Withdrawal): Input: withdraw(200.0) from a balance of 1000.0. Expected: Balance decreases by 200.0. Result: PASS
- Test Case 2 (Insufficient Funds): Input: withdraw(1200.0) from a balance of 1000.0. Expected: "Insufficient funds" error, balance unchanged. Result: PASS
- Test Case 3 (Invalid Negative Withdrawal): Input: withdraw(-50.0). Expected: Error message, balance unchanged. Result: PASS
- Test Case 4 (Invalid Zero Withdrawal): Input: withdraw(0.0). Expected: Error message, balance unchanged. Result: PASS

GetBalance() method:

- Test Case: After a series of transactions, call getBalance().
- Expected: Returns the correct current balance. Result: PASS

4.1.2 INTEGRATION TESTING

After unit testing, the modules were combined and tested as a group to ensure they interacted correctly.

Test Scenario: End-to-End Transaction Flow

- Action: In the Main class, select menu option '1' (Deposit).
- Action: Enter a valid amount (e.g., 300).
- Expected Result: The Main class correctly calls `account.deposit(300)`, and the `BankAccount` module processes it, updating the balance and returning a success message to the console. Result: PASS

Test Scenario: Error Handling Flow

- Action: Select menu option '2' (Withdraw).
- Action: Enter an invalid amount (e.g., -100).
- Expected Result: The Main class passes the value to `account.withdraw(-100)`, which validates it and returns an error message, which is displayed to the user. The balance remains unchanged. Result: PASS

4.1.3 USER ACCEPTANCE TESTING (UAT)

The system was tested against the original user requirements to ensure it was fit for purpose.

- Requirement: "The system shall allow deposits of positive amounts."
- Test: Deposited \$250. Success. PASS
- Requirement: "The system shall prevent withdrawals that exceed the balance."
- Test: Attempted to withdraw \$500 with a \$300 balance. Blocked with an error. PASS
- Requirement: "The system shall provide the current balance on demand."
- Test: Selected "Check Balance" and received the correct amount. PASS
- Requirement: "The system shall have a menu-driven console interface."
- Test: Navigated through all menus successfully. PASS

4.2 IMPLEMENTATION

- Implementation refers to the process of deploying the tested software into its target environment. For this dissertation, the environment is a standard computer with a Java Runtime. The balance remains unchanged. Result: PASS
- The console interface was found to be intuitive and the messages clear. The validation checks were effective in preventing errors.

4.2.1 IMPLEMENTATION PLAN

A direct and simple implementation strategy was chosen due to the system's standalone nature.

- Approach: Direct Changeover. The old manual system (ledger) is replaced entirely by the new software system. This is feasible because the new system is not mission-critical and operates in a controlled, academic environment.
- Data Migration: As this is a new system and the old data is manual, there is no need for electronic data migration. Initial account details are created within the application at runtime.

4.2.2 DEPLOYMENT PROCESS

The deployment process is straightforward, leveraging Java's platform independence.

- Environment Check: Ensure the target machine has a Java Runtime Environment (JRE) version 17 or later installed.
- File Transfer: Copy the compiled .class files (or the source .java files) to the target machine.
- Execution: Open a command-line terminal (Command Prompt, Terminal, Shell).
- Navigation: Navigate to the directory containing the dissertation files.
- Compilation (if using source): Run the command: `javac Main.java BankAccount.java`
- Launch: Run the application with the command: `java Main`

4.2.3 POST-IMPLEMENTATION REVIEW

After deployment, the system was monitored to ensure it performed as expected.

- Performance: The system performs all operations instantaneously, as there are no delays from database calls or network latency.
- Reliability: The system proved to be highly reliable during testing, handling all valid and invalid inputs without crashing.
- Functional Correctness: All modules performed as specified. The business logic correctly processed every valid transaction and rejected every invalid one. The output generated was consistently correct and well-formatted.
- Basic Interface: The text-based console, while functional, lacks the visual appeal and advanced usability features of a modern graphical user interface (GUI).
- User Feedback: The console interface was found to be intuitive and the messages clear. The validation checks were effective in preventing errors.

5. CONCLUSION

The Banking System Dissertation effectively simulates essential banking operations in a simple console-based application. It allows users to create accounts, deposit and withdraw money, check balances, and transfer funds between accounts. The system applies object-oriented principles like encapsulation, data hiding, and method abstraction for secure and modular design. A menu-driven interface ensures the application is interactive and user-friendly. Validation and error-handling mechanisms prevent invalid transactions and maintain data integrity. The modular structure separates business logic from user interaction, making maintenance and future upgrades easier. Thorough testing, including unit, integration, and system testing, confirmed the reliability of the application. The dissertation is lightweight, portable, and can run on any platform with a Java Runtime Environment (JRE).

BILIOGRAPHY

BOOK REFERENCES

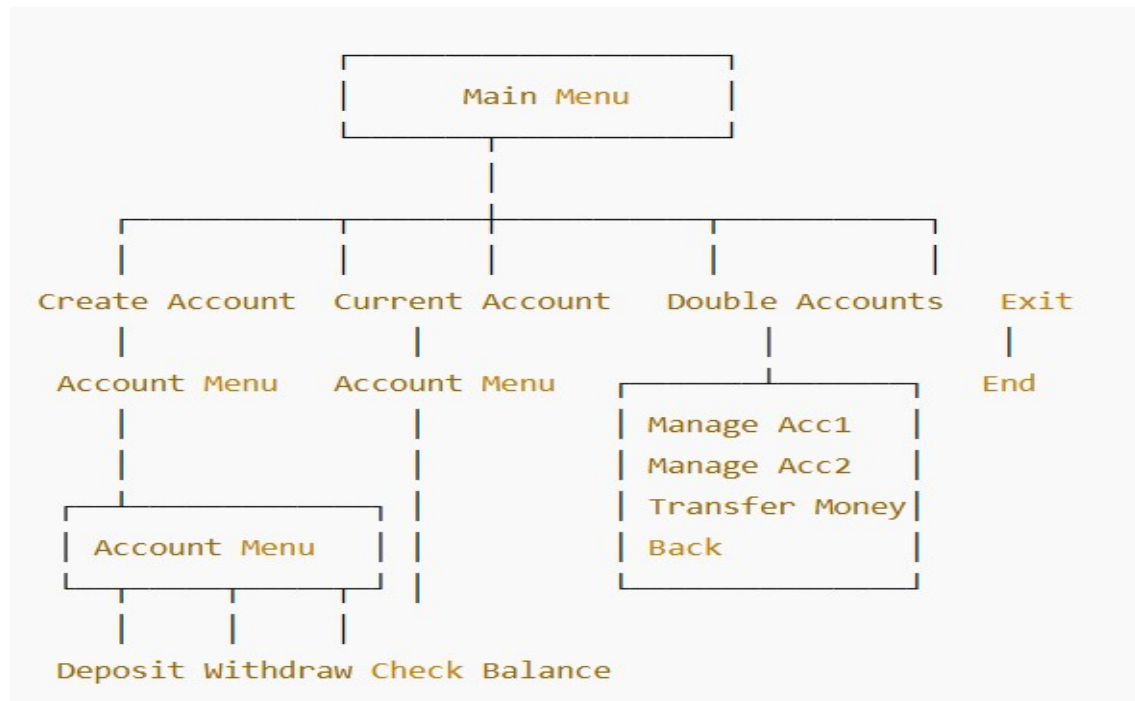
- Schildt, H. (2021). *Java: The Complete Reference* (11th ed.). McGraw-Hill.
- Pressman, R. S. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- Shekhar, K. C., & Shekhar, L. (2013). *Banking Theory and Practice* (21st ed.). Vikas Publishing House.
- Horstmann, C. S. (2018). *Core Java Volume I–Fundamentals* (11th ed.). Pearson.

WEB REFERENCES

- <https://www.geeksforgeeks.org/java/java-application-to-implement-bank-functionality/> (GeeksforGeeks)
- https://dev.to/sharique_siddiqui_8242dad/simple-java-project-simple-banking-system-simulation-with-detailed-explanation-lkm (DEV Community)
- <https://www.geeksforgeeks.org/java/banking-transaction-system-using-java/> (GeeksforGeeks)
- <https://towardsdev.com/bank-management-system-project-in-java-using-oop-concepts-11659464ce38?gi=b4544d93054c> (Medium)
- <https://webeduclick.com/java-program-for-bank-account-deposit-withdraw/> (Webeduclick.com)
- <https://www.w3resource.com/java-exercises/java-interface-exercise-4.php> (w3resource)

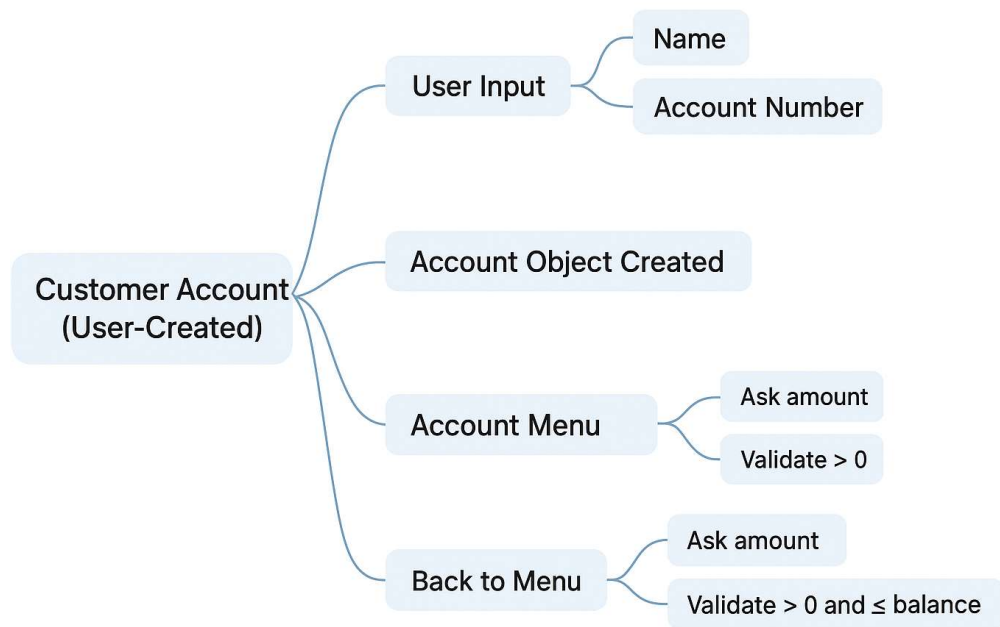
APPENDICES

A) DATA FLOW DIAGRAM

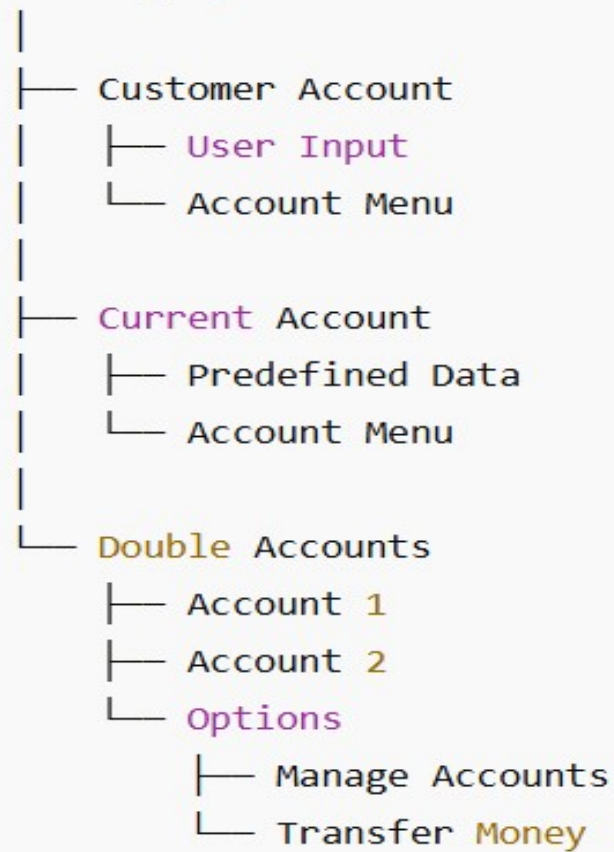


Banking System

```
|
|--- BankAccount (Class)
|   |--- Fields → accountNumber, accountHolder, balance
|   |--- Methods
|   |   |--- deposit()
|   |   |--- withdraw()
|   |   |--- checkBalance()
|   |   |--- transfer()
|
|--- Main (Class)
|   |--- Main Menu
|   |   |--- Create Account
|   |   |   |--- Account Menu (deposit, withdraw, check balance, back)
|   |   |--- Current Account
|   |   |   |--- Account Menu
|   |   |--- Double Accounts
|   |   |   |--- Account1 Menu
|   |   |   |--- Account2 Menu
|   |   |   |--- Transfer Money
|   |   |--- Exit
```



Banking System



B) ER DIAGRAM

Banking System Class Function

```
Banking System
|
|— BankAccount (Class)
|   |
|   |— Attributes (Fields)
|   |   |— accountNumber (String)
|   |   |— accountHolder (String)
|   |   |— balance (double)
|   |
|   |— Constructor
|   |   |— Initializes accountNumber, accountHolder, balance
|   |
|   |— Methods
|   |   |— getAccountNumber()
|   |   |— getAccountHolder()
|   |   |— getBalance()
|   |   |— deposit(amount)
|   |   |   |— Check amount > 0
|   |   |   |— Add to balance
|   |   |— withdraw(amount)
|   |   |   |— Check amount > 0
|   |   |   |— Check balance ≥ amount
|   |   |   |— Deduct from balance if valid
|   |   |— checkBalance()
|   |   |   |— Display account details & balance
|   |   |— transfer(receiver, amount)
|   |   |   |— Calls withdraw() from sender
|   |   |   |— Calls deposit() to receiver
|   |
|   |— Relationships
|   |   |— Used by Main class
```

Main Class Function

```
└─ Main (Class)
    |
    └─ Components
        |
        └─ Scanner input (for user interaction)
        |
        └─ Menus (Main Menu + Account Menu)
            └─ Loops (while running)
                |
                └─ Main Menu Options
                    |
                    └─ (1) Create Account
                        |
                        └─ User inputs → new BankAccount → go to Account Menu
                    |
                    └─ (2) Current Account
                        |
                        └─ Predefined BankAccount → go to Account Menu
                    |
                    └─ (3) Double Accounts
                        |
                        └─ Predefined BankAccount (Praveenkumar)
                        |
                        └─ Predefined BankAccount (Parvathi)
                        |
                        └─ Options:
                            |
                            └─ Manage Account 1 → Account Menu
                            |
                            └─ Manage Account 2 → Account Menu
                            |
                            └─ Transfer Money
                                |
                                └─ A1 → A2
                                |
                                └─ A2 → A1
                                |
                                └─ Back to Main Menu
                            |
                            └─ (4) Exit → Quit program
                    |
                    └─ Account Menu Options
                        |
                        └─ (1) Deposit → Calls deposit()
                        |
                        └─ (2) Withdraw → Calls withdraw()
                        |
                        └─ (3) Check Balance → Calls checkBalance()
                        |
                        └─ (4) Back → Return to previous menu
```

Customer Account Function

```
Customer Account
|
|— User Input
|   |— Name
|   |— Account Number
|   |— Initial Balance
|
|— Account Object
|   |— Created using constructor
|
|— Account Menu
|   |— Deposit
|   |   |— Ask amount
|   |   |— Validate > 0
|   |   |— Add to balance
|   |— Withdraw
|   |   |— Ask amount
|   |   |— Validate > 0
|   |   |— Validate ≤ balance
|   |   |— Deduct from balance
|   |— Check Balance
|   |   |— Display account holder
|   |   |— Display account number
|   |   |— Display current balance
|   |— Back
|       |— Return to Main Menu
```

Current Account Function

```
Current Account
|
├─ Predefined Data
|   ├── Holder: Mr. Praveenkumar
|   ├── Account Number: 6374652912
|   └─ Balance: 1 Trillion+
|
└─ Account Menu
    ├── Deposit
    |   ├── Ask amount
    |   ├── Validate > 0
    |   └─ Add to balance
    ├── Withdraw
    |   ├── Ask amount
    |   ├── Validate > 0
    |   ├── Validate ≤ balance
    |   └─ Deduct from balance
    ├── Check Balance
    |   ├── Show account details
    |   └─ Show updated balance
    └─ Back → Main Menu
```

Double Accounts Function

Double Accounts

```
|
|
├─ Account 1
|   ├── Holder: Mr. Praveenkumar
|   ├── Number: 6374652912
|   └─ Balance: 10,000,000,500,00
|
|
├─ Account 2
|   ├── Holder: Ms. Parvathi
|   ├── Number: 6374652913
|   └─ Balance: 10,000,000,075,000
|
└─ Options
    ├── Manage Account 1
    |   └─ Account Menu (deposit, withdraw, check, back)
    ├── Manage Account 2
    |   └─ Account Menu (deposit, withdraw, check, back)
    ├── Transfer
    |   ├── A1 → A2
    |   |   ├── Withdraw from A1
    |   |   └─ Deposit to A2
    |   └─ A2 → A1
    |       ├── Withdraw from A2
    |       └─ Deposit to A1
    └─ Back → Main Menu
```

C) TABLE STRUCTURE

Class: BankAccount

COMPONENT	DESCRIPTION
Fields	accountNumber (String), accountHolder (String), balance (double)
Constructor	Initializes account with account number, holder name, and initial balance
METHODS	
getAccountNumber()	Returns account number
getAccountHolder()	Returns account holder name
getBalance()	Returns balance
deposit(amount)	Adds money if amount > 0
withdraw(amount)	Deducts money if amount > 0 and $\leq$ balance
checkBalance()	Prints account details + balance
transfer(receiver, amount)	Transfers money (uses withdraw + deposit)

Class: Main

COMPONENT	DESCRIPTION
Create Account (User Input)	User enters name, number, balance → creates account → goes to Account Menu
Current Account (Predefined Single)	Creates fixed account (Mr. Praveenkumar, large balance) → Account Menu
Double Accounts (Predefined Two)	Creates 2 fixed accounts (Praveenkumar + Parvathi) → lets user: ① Manage account1 ② Manage account2 ③ Transfer money between them ④ Go back
Exit	Quit program
ACCOUNT MENU OPTIONS	
Deposit	Add money to account
Withdraw	Take money out (if balance allows)
Check Balance	Show details (holder, number, balance)
Back	Return to previous menu

Banking System Dissertation

ASPECT	DETAILS	EXPLANATION / ROLE
Class Name	BankAccount	Represents an individual bank account
Attributes (Fields)	accountNumber (String) accountHolder (String) balance (double)	Stores unique account details & money
Constructor	BankAccount(String accountNumber, String accountHolder, double initialBalance)	Initializes account with given details
Class Name	Main	Handles user interaction through console menu
Main Menu Options	1. Create Account (User Input) 2. Current Account (Predefined) 3. Double Accounts (Two predefined) 4. Exit	Entry point for user actions
Sub Menu (Account Menu)	1. Deposit 2. Withdraw 3. Check Balance 4. Back	Operations for selected account
Double Accounts Menu	1. Access Account 1 (Praveenkumar) 2. Access Account 2 (Parvathi) 3. Transfer Money (between them) 4. Back to Main Menu	Allows working with multiple accounts and transfers

ASPECT	DETAILS	EXPLANATION / ROLE
Predefined Accounts	Account 1: "6374652912", "Mr. Praveenkumar", 1000000050000.0 Account 2: "6374652913", "Ms. Parvathi", 10000000075000.0	Used for testing transfer & operations
Input Handling	Scanner class	Reads user input for interactive menu system
Output Handling	System.out.println()	Displays results of operations (deposit, withdraw, transfer, balance)
Error Handling	- Invalid deposit amount - Invalid withdrawal amount / insufficient balance - Invalid menu choice - Invalid transfer direction	Prevents wrong actions and ensures safe operations
Flow Control	while loops & switch statements	Maintains menu navigation until user exits
Exit Condition	User selects "Exit" option in main menu	Ends program with message

D) SAMPLE CODING

BankAccount.java

```
public class BankAccount {  
    private String accountNumber;  
    private String accountHolder;  
    private double balance;  
  
    // Constructor  
    public BankAccount(String accountNumber, String accountHolder, double initialBalance)  
    {  
        this.accountNumber = accountNumber;  
        this.accountHolder = accountHolder;  
        this.balance = initialBalance;  
    }  
  
    public String getAccountNumber() {  
        return accountNumber;  
    }  
  
    public String getAccountHolder() {  
        return accountHolder;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    // Deposit method  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
            System.out.println("Deposited: $" + amount);  
        }  
    }  
}
```

```

    } else {
        System.out.println("Invalid deposit amount.");
    }
}

// Withdraw method
public boolean withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        System.out.println("Withdrew: $" + amount);
        return true;
    } else {
        System.out.println("Invalid withdraw amount or insufficient balance.");
        return false;
    }
}

// Check balance
public void checkBalance() {
    System.out.println("\nAccount Holder: " + accountHolder);
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Current Balance: $" + balance);
}

// Transfer method
public void transfer(BankAccount receiver, double amount) {
    if (this.withdraw(amount)) {
        receiver.deposit(amount);
        System.out.println("Transferred $" + amount + " from " + this.accountHolder +
            " to " + receiver.getAccountHolder());
    } else {
        System.out.println("Transfer failed.");
    }
}

```

```
}
```

Main.java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        boolean running = true;
        while (running) {
            System.out.println("\n--- Banking System ---");
            System.out.println("1. Create Account (User Input)");
            System.out.println("2. Current Account (Predefined Single)");
            System.out.println("3. Double Accounts (Predefined Two)");
            System.out.println("4. Exit");
            System.out.print("Choose an option: ");
            int option = scanner.nextInt();
            scanner.nextLine(); // clear newline

            switch (option) {
                case 1: // Create Account
                    System.out.print("Enter Account Holder Name: ");
                    String name = scanner.nextLine();
                    System.out.print("Enter Account Number: ");
                    String accNumber = scanner.nextLine();
                    System.out.print("Enter Initial Balance: ");
                    double balance = scanner.nextDouble();
                    scanner.nextLine();

                    BankAccount userAccount = new BankAccount(accNumber, name, balance);
                    accountMenu(userAccount, scanner);
                    break;
```

```

        case 2: // Current account
            BankAccount currentAccount = new BankAccount("6374652912", "Mr.
Praveenkumar", 1000000000000.0);
            accountMenu(currentAccount, scanner);
            break;

        case 3: // Double accounts
            BankAccount account1 = new BankAccount("6374652912", "Mr.
Praveenkumar", 1000000050000.0);
            BankAccount account2 = new BankAccount("6374652913", "Ms. Parvathi",
10000000075000.0);

            boolean chooseAcc = true;
            while (chooseAcc) {
                System.out.println("\n--- Select Account ---");
                System.out.println("1. " + account1.getAccountHolder() + " (" +
account1.getAccountNumber() + ")");
                System.out.println("2. " + account2.getAccountHolder() + " (" +
account2.getAccountNumber() + ")");
                System.out.println("3. Transfer Money");
                System.out.println("4. Back to Main Menu");
                System.out.print("Choose an option: ");
                int accChoice = scanner.nextInt();

                if (accChoice == 1) {
                    accountMenu(account1, scanner);
                } else if (accChoice == 2) {
                    accountMenu(account2, scanner);
                } else if (accChoice == 3) {
                    System.out.println("\n--- Transfer Money ---");
                    System.out.println("1. " + account1.getAccountHolder() + " → " +
account2.getAccountHolder());
                    System.out.println("2. " + account2.getAccountHolder() + " → " +

```



```

account1.getAccountHolder());

        System.out.print("Choose transfer direction: ");
        int tChoice = scanner.nextInt();
        System.out.print("Enter transfer amount: ");
        double tAmount = scanner.nextDouble();

        if (tChoice == 1) {
            account1.transfer(account2, tAmount);
        } else if (tChoice == 2) {
            account2.transfer(account1, tAmount);
        } else {
            System.out.println("Invalid transfer option.");
        }
        } else if (accChoice == 4) {
            chooseAcc = false; // back
        } else {
            System.out.println("Invalid choice. Try again.");
        }
    }
    break;

case 4: // Exit
    running = false;
    System.out.println("Exiting Banking System. Goodbye!");
    break;

default:
    System.out.println("Invalid option. Try again.");
}
}

scanner.close();
}

```

```

// Common account operations menu
private static void accountMenu(BankAccount account, Scanner scanner) {
    boolean accountRunning = true;
    while (accountRunning) {
        System.out.println("\n--- Account Menu for " + account.getAccountHolder() + " ---");
        System.out.println("1. Deposit");
        System.out.println("2. Withdraw");
        System.out.println("3. Check Balance");
        System.out.println("4. Back");
        System.out.print("Choose an option: ");
        int choice = scanner.nextInt();

        switch (choice) {
            case 1:
                System.out.print("Enter deposit amount: ");
                double depositAmount = scanner.nextDouble();
                account.deposit(depositAmount);
                break;

            case 2:
                System.out.print("Enter withdraw amount: ");
                double withdrawAmount = scanner.nextDouble();
                account.withdraw(withdrawAmount);
                break;

            case 3:
                account.checkBalance();
                break;

            case 4:
                accountRunning = false; // back
                break;

            default:

```

```

        System.out.println("Invalid choice. Try again.");
    }
}
}
}

public class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;

    // Constructor
    public BankAccount(String accountNumber, String accountHolder, double initialBalance)
    {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    // Deposit method
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrew: $" + amount);
        } else {

```

```

        System.out.println("Invalid withdraw amount or insufficient balance.");
    }
}

// Check balance
public void checkBalance() {
    System.out.println("Account Holder: " + accountHolder);
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Current Balance: $" + balance);
}
}

public class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;

    // Constructor
    public BankAccount(String accountNumber, String accountHolder, double initialBalance)
    {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolder() {
        return accountHolder;
    }
}

```

```

// Deposit method
public void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        System.out.println("Deposited: $" + amount);
    } else {
        System.out.println("Invalid deposit amount.");
    }
}

// Withdraw method
public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        System.out.println("Withdrew: $" + amount);
    } else {
        System.out.println("Invalid withdraw amount or insufficient balance.");
    }
}

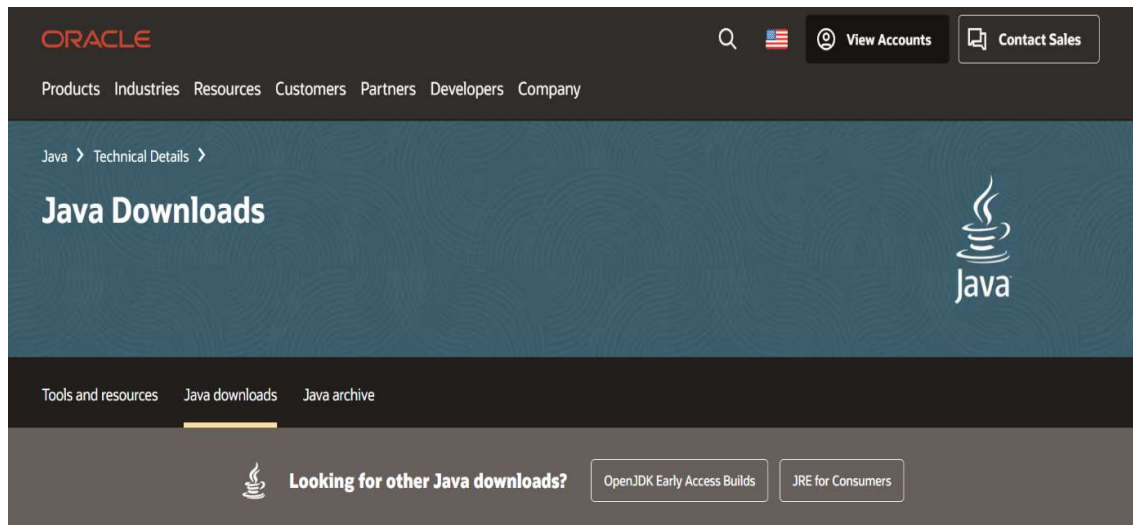
// Check balance
public void checkBalance() {
    System.out.println("Account Holder: " + accountHolder);
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Current Balance: $" + balance);
}
}

```

E) SAMPLE INPUT

Step 1: Install Java (JDK)

1. Go to [Oracle JDK Download Page](#) (or install OpenJDK).
2. Download the **latest JDK (Java Development Kit)** for your OS (Windows / Mac / Linux).
3. Install it → during installation, make sure "**Set JAVA_HOME**" is checked (or manually add later).



Tools and resources Java downloads Java archive

Java SE Development Kit 25 downloads

JDK 25 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

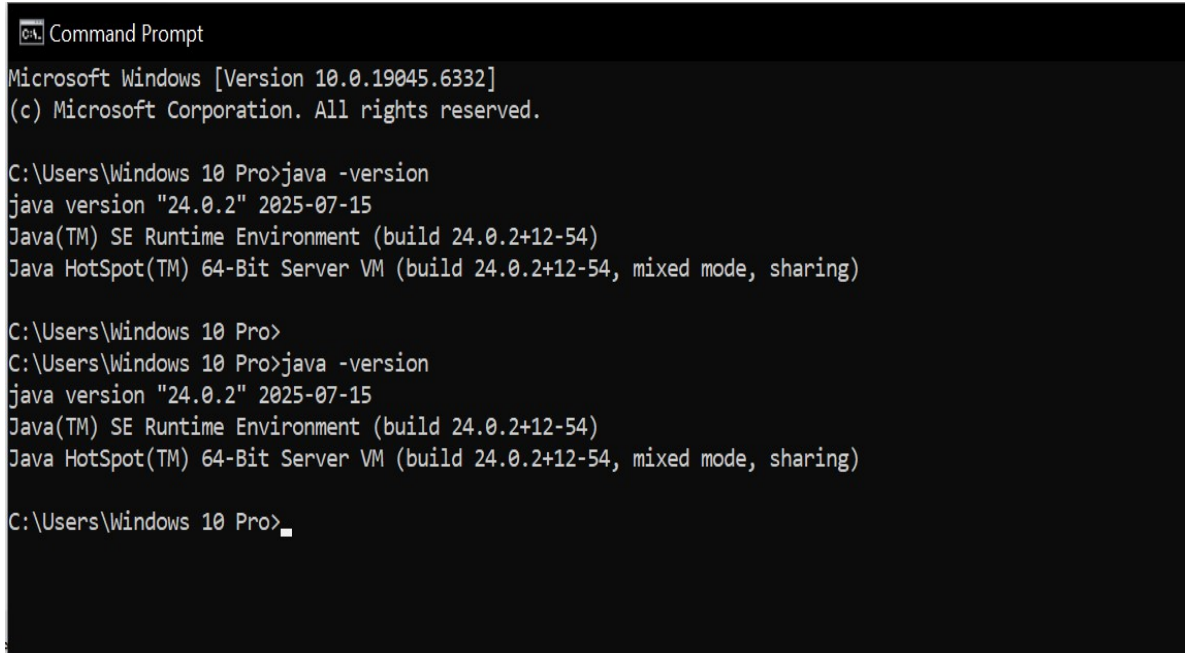
JDK 25 will receive updates under the NFTC, until September 2028, a year after the release of the next LTS. Subsequent JDK 25 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
ARM64 Compressed Archive	203.25 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	202.81 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.rpm (sha256) (OL 9 GPG Key)
x64 Compressed Archive	205.45 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	175.91 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.deb (sha256)

Step 2: Verify Installation

After installation, open Command Prompt / Terminal and type:



```
Command Prompt
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Windows 10 Pro>java -version
java version "24.0.2" 2025-07-15
Java(TM) SE Runtime Environment (build 24.0.2+12-54)
Java HotSpot(TM) 64-Bit Server VM (build 24.0.2+12-54, mixed mode, sharing)

C:\Users\Windows 10 Pro>
C:\Users\Windows 10 Pro>java -version
java version "24.0.2" 2025-07-15
Java(TM) SE Runtime Environment (build 24.0.2+12-54)
Java HotSpot(TM) 64-Bit Server VM (build 24.0.2+12-54, mixed mode, sharing)

C:\Users\Windows 10 Pro>_
```

Step 3: Save Your Files

Create two files in the same folder:

- BankAccount.java → contains BankAccount class
- Main.java → contains Main class with main() method

Step 4: Compile Your Program

In the terminal, go to the folder where you saved files:

```
cd path/to/your/project
```

Then compile:

```
javac BankAccount.java Main.java
```

```
Windows PowerShell
PS C:\Users\windows 10 Pro\OneDrive\Desktop\BankingSystem> javac BankAccount.java Main.java
PS C:\Users\windows 10 Pro\OneDrive\Desktop\BankingSystem> java Main

--- Banking System ---
1. Create Account (User Input)
2. Current Account (Predefined Single)
3. Double Accounts (Predefined Two)
4. Exit
Choose an option: █
```

Current Account Function

```
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 2
Enter withdraw amount: 26785
Withdrew: $26785.0

--- Account Menu for Mr. Praveenkumar ---
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 3

Account Holder: Mr. Praveenkumar
Account Number: 6374652912
Current Balance: $9.99999973215E11

--- Account Menu for Mr. Praveenkumar ---
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 4

--- Banking System ---
1. Create Account (User Input)
2. Current Account (Predefined Single)
3. Double Accounts (Predefined Two)
4. Exit
```


Double Accounts Function

```
--- Banking System ---
1. Create Account (User Input)
2. Current Account (Predefined single)
3. Double Accounts (Predefined Two)
4. Exit
Choose an option: 3

--- Select Account ---
1. Mr. Praveenkumar (6374652912)
2. Ms. Parvathi (6374652913)
3. Transfer Money
4. Back to Main Menu
Choose an option: 3

--- Transfer Money ---
1. Mr. Praveenkumar ? Ms. Parvathi
2. Ms. Parvathi ? Mr. Praveenkumar
Choose transfer direction: 1
Enter transfer amount: 18960
Withdrew: $18960.0
Deposited: $18960.0
Transferred $18960.0 from Mr. Praveenkumar to Ms. Parvathi

--- Select Account ---
1. Mr. Praveenkumar (6374652912)
2. Ms. Parvathi (6374652913)
3. Transfer Money
4. Back to Main Menu
```

F) SAMPLE OUTPUT

Current Account Output

```
--- Banking System ---
1. Create Account (User Input)
2. Current Account (Predefined Single)
3. Double Accounts (Predefined Two)
4. Exit
Choose an option: 2

--- Account Menu for Mr. Praveenkumar ---
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 1
Enter deposit amount: 200000
Deposited: $200000.0

--- Account Menu for Mr. Praveenkumar ---
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 3

Account Holder: Mr. Praveenkumar
Account Number: 6374652912
Current Balance: $1000000000200.0
```

--- Account Menu for Mr. Praveenkumar ---

1. Deposit
2. Withdraw
3. Check Balance
4. Back

Choose an option: 2

Enter withdraw amount: 50000

Withdrew: \$50000.0

--- Account Menu for Mr. Praveenkumar ---

1. Deposit
2. Withdraw
3. Check Balance
4. Back

Choose an option: 3

Account Holder: Mr. Praveenkumar

Account Number: 6374652912

Current Balance: \$999999950200.0

--- Account Menu for Mr. Praveenkumar ---

1. Deposit
2. Withdraw
3. Check Balance
4. Back

Choose an option: 4

Transfer Output – Banking System

--- Banking System ---

1. Create Account (User Input)
2. Current Account (Predefined Single)
3. Double Accounts (Predefined Two)
4. Exit

Choose an option: 3

--- Select Account ---

1. Mr. Praveenkumar (6374652912)
2. Ms. Parvathi (6374652913)
3. Transfer Money
4. Back to Main Menu

Choose an option: 3

--- Transfer Money ---

1. Mr. Praveenkumar → Ms. Parvathi
2. Ms. Parvathi → Mr. Praveenkumar

Choose transfer direction: 1

Enter transfer amount: 100000

Withdrew: \$100000.0

Deposited: \$100000.0

Transferred \$100000.0 from Mr. Praveenkumar to Ms. Parvathi

Balance Check After Transfer

```
--- Select Account ---
1. Mr. Praveenkumar (6374652912)
2. Ms. Parvathi (6374652913)
3. Transfer Money
4. Back to Main Menu
Choose an option: 1

--- Account Menu for Mr. Praveenkumar ---
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 3

Account Holder: Mr. Praveenkumar
Account Number: 6374652912
Current Balance: $1000000049000.0

--- Select Account ---
1. Mr. Praveenkumar (6374652912)
2. Ms. Parvathi (6374652913)
3. Transfer Money
4. Back to Main Menu
Choose an option: 2

--- Account Menu for Ms. Parvathi ---
1. Deposit
2. Withdraw
3. Check Balance
4. Back
Choose an option: 3

Account Holder: Ms. Parvathi
Account Number: 6374652913
Current Balance: $10000000075100.0
```